PROMPTZAL V-3 LFP

Manual técnico

JONATHAN ESTUARDO ZORIN HERNANDEZ

1. **CONTEXTO DEL PROYECTO:** Construir un analizador sintáctico fase que toma la secuencia de tokens y verifica que formen un programa válido según la gramática libre de contexto del lenguaje. Mientras lo hace se construye el árbol de sintaxis abstracta (AST) y la tabla de símbolos, validará el significado del programa y, finalmente, generará el promptzal: el prompt en texto, listo para copiar y pegar en Claude Code o en cualquier herramienta de IA.
2. **PRYECTO BASADO EN:** Programación Java Swing · Maven · JFlex · Parser descendente recursivo.

3. **ESTRUCTURA:**
   - **src/main/jflex/PromptZal.jflex** – especificación léxica (JFlex).
   - **`parser/Parser.java`** – un método por no terminal; recuperación en modo pánico.
   - **`ast/Ast.java`** – nodos del AST.
   - **`semantic`** – tabla de símbolos y validaciones.
   - **`generator/GeneradorPrompt.java`** – promptzal.
   - **`report/Reportes.java`** – reportes HTML y AST.
   - **`gui`** – IDE (editor con resaltado, errores, símbolos, AST, tokens, promptzal).
   - **`.pz`**, promptzal generado y reportes de muestra.

4. TECNOLOGIAS Y REQUISITOS

JDK 11 o superior. Maven: administra la compilación y plugins. JFlex: genera el lexer a partir de src/main/jflex/PromptZal.jflex.

Swing: construye la interfaz gráfica. No se hay un backend web ni una base de datos: la lógica de compilación se ejecuta localmente dentro de la aplicación Java.

5. INICIAR EL PROGRAMA

En NetBeans: Archivo → Abrir proyecto → seleccionar la carpeta PromptZal que contiene pom.xml → Clean and Build → Run. La primera compilación puede requerir internet para descargar el plugin y dependencias de Maven.

6. FLUJO DE COMPILACION



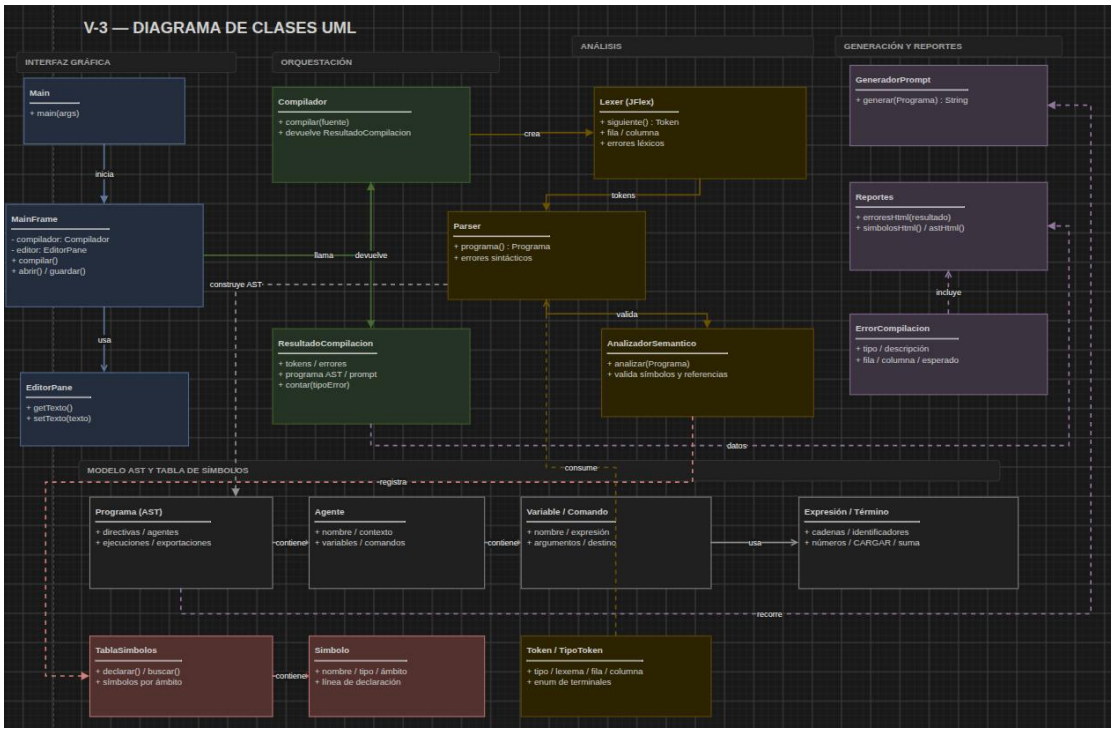
La clase Compilador.compilar (String fuente) coordina las fases: crea Lexer con un StringReader, obtiene tokens hasta EOF, recoge errores léxicos, crea Parser con los tokens, llama a programa(), ejecuta AnalizadorSemantico y finalmente GeneradorPrompt.generar(). Los errores se almacenan en ResultadoCompilacion.

7. COMPONENTES Y RESPONSABILIDADES DEL SISTEMA

Paquete / clase	Responsabilidad y conexión
promptzal.Main	Punto de entrada; abre MainFrame.
gui.MainFrame	Ventana principal. Conecta acciones del menú y botones con abrir, guardar, compilar, guardar prompt y reportes.
gui.EditorPane / Tema	Edición y apariencia de la interfaz.
compiler.Compilador	Orquesta lexer → parser → semántica → generador.
lexer.PromptZal.jflex	Especificación fuente de JFlex. Genera promptzal.lexer.Lexer en target/generated-sources/jflex.
lexer.Token / TipoToken	Estructura y enumeración de tokens que consume el parser.
parser.Parser	Parser descendente recursivo escrito a mano; construye el AST y recopila errores sintácticos.
ast.Ast	Clases de nodos que representan programa, agentes, variables, comandos y expresiones.

semantic.AnalizadorSemantico / TablaSimbolos / Simbolo	Comprueba declaraciones, ámbitos y referencias; registra agentes y variables.
generator.GeneradorPrompt	Recorre el AST y redacta el prompt final.
report.Reportes	Genera representaciones HTML y texto de AST.
util.ErrorCompilacion / compiler.ResultadoCompilacion	Transportan los errores y resultados entre las fases.

8. DIAGRAMA DE CLASES



9. INTEGRACION DE JFLEX

El archivo fuente del lexer es src/main/jflex/PromptZal.jflex. Maven lo procesa y genera Lexer.java dentro de target/generated-sources/jflex/promptzal/lexer/. No se debe mantener otra clase promptzal.lexer.Lexer manual dentro de src/main/java, porque el compilador reportará duplicate class. Si aparece el error, elimina únicamente la copia manual y ejecuta Clean and Build; conserva PromptZal.jflex, Token.java y TipoToken.java.

10. AST, TABLA DE SIMBOLOS Y SEMANTICA

El parser construye el AST a medida que reconoce la gramática. Después, AnalizadorSemantico analiza el programa y expone la tabla mediante getTabla(). El diseño del proyecto registra agentes y variables, con tipo, ámbito y línea. Entre las verificaciones descritas en el proyecto están la declaración previa de identificadores en el mismo agente, la unicidad de nombres de agentes, la existencia de agentes usados en EJECUTAR y la validez de variables incluidas en EXPORTAR.

11. ERRORES Y RECUPERACION

Los errores se representan mediante ErrorCompilacion e incluyen categoría, descripción y ubicación. El parser mantiene una lista de errores y aplica recuperación para continuar tras errores sintácticos. Resultado Compilacion reúne tokens, programa AST, tabla, errores y prompt para que MainFrame actualice sus paneles.

12. GENERACION DEL PROMPT Y RESPORTES

GeneradorPrompt genera el texto a partir del AST y las ejecuciones declaradas. MainFrame permite guardar el prompt en .md o .txt. Reportes genera HTML para errores, tabla de símbolos y AST. Los reportes se guardan en una carpeta seleccionada por el usuario o en la ubicación predeterminada que gestione la interfaz.

13. COMPILACION, DIAGNOSTICO Y MANTENIMINETO

mvn clean package\n# o desde NetBeans: Clean and Build Si falla la compilación: (1) comprobar JDK 11 o superior; (2) revisar el error completo en Output; (3) confirmar que JFlex genera una sola clase Lexer; (4) limpiar target mediante mvn clean; (5) volver a construir. No editar directamente el Lexer generado en target: los cambios deben hacerse en PromptZal.jflex.

14. BACKEND

Este proyecto no utiliza un backend remoto: MainFrame llama al objeto Compilador dentro de la misma aplicación. Compilador invoca directamente las clases del lexer, parser, semántica y generador. No se encontró una API HTTP ni una conexión a base de datos en la estructura analizada; por ello, la integración se describe como una arquitectura local por capas, no como cliente-servidor.